

Homework 4 - LSTM

Goal: Build a Long Short Term Memory (LSTM) neural network to generate music by generating sequence of notes in a music. (10 points)

Data: A classical piano composition by Mozart is used to train the neural network. Use the sample MIDI (Musical Instrument Digital Interface) file, `mozart.mid`. A MIDI music file consists of several notes that have attributes like start time, end time, pitch and velocity. The python package `pretty_midi` could be used to extract this information from the music file.

Pre-defined functions: The provided functions below generate input data from MIDI files, generate new sequence of notes from a given sequence, and to create a MIDI file from the generated sequence.

1. Historical input data of a particular sequence length along with the corresponding output can be generated using the function `generate_input_data`. The converter parses the MIDI file and generates a list of all notes and chords in the file. Each chord is appended by encoding the ID of every note in the chord together into a single string. These encodings allow the decoding of the output generated by the LSTM network into correct notes and chords. Once the sequential list of all notes and chords is available, historical input data is generated. Finally, the network input data is normalized and the network output data is converted into a categorical array.
2. The function `generate_sequence` could be used to generate a new sequence.
3. The function `create_midi` will convert any given sequence of notes into a MIDI file.

```
In [1]: pip install pretty_midi
```

```

Collecting pretty_midi
  Downloading pretty_midi-0.2.11.tar.gz (5.6 MB)
    _____ 5.6/5.6 MB 55.4 MB/s eta 0:00:00
    Preparing metadata (setup.py) ... done
Requirement already satisfied: numpy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from pretty_midi) (2.0.2)
Collecting mido>=1.1.16 (from pretty_midi)
  Downloading mido-1.3.3-py3-none-any.whl.metadata (6.4 kB)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from pretty_midi) (1.17.0)
Requirement already satisfied: importlib_resources in /usr/local/lib/python3.12/dist-packages (from pretty_midi) (6.5.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from mido>=1.1.16->pretty_midi) (26.0)
Downloading mido-1.3.3-py3-none-any.whl (54 kB)
    _____ 54.6/54.6 kB 8.0 MB/s eta 0:00:00
Building wheels for collected packages: pretty_midi
  Building wheel for pretty_midi (setup.py) ... done
  Created wheel for pretty_midi: filename=pretty_midi-0.2.11-py3-none-any.whl size=5595886 sha256=4ffbc2d5c85c7177595f8fd6aed192ac2275105b4bb30bbd023145a9ca60347
  Stored in directory: /root/.cache/pip/wheels/f4/ad/93/a7042fe12668827574927ade9deec7f29aad2a1001b1501882
Successfully built pretty_midi
Installing collected packages: mido, pretty_midi
Successfully installed mido-1.3.3 pretty_midi-0.2.11

```

```

In [2]: # Import necessary libraries
from music21 import converter, instrument, note, chord, stream
import tensorflow, pretty_midi

import shutil
from google.colab import files
import os

from pathlib import Path
from dotenv import load_dotenv
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import random_split, DataLoader, TensorDataset
from IPython.display import display, Image
import time
import copy
from datetime import datetime

# Loss plots and deliverable images are saved under this folder (single source of t
PLOT_OUTPUT_DIR = "Plot JPGs"

```

```

In [3]: def generate_input_data(song_path):

        notes = []

```

```

midi = converter.parse(song_path)
print("Parsing %s" % song_path)

# use flat notes only
notes_to_parse = midi.flat.notes

for element in notes_to_parse:
    if isinstance(element, note.Note):
        notes.append(str(element.pitch))
    elif isinstance(element, chord.Chord):
        notes.append('.'.join(str(n) for n in element.normalOrder))

sequence_length = 100

# get all pitch names
pitchnames = sorted(set(item for item in notes))

# create a dictionary to map pitches to integers
note_to_int = dict((note, number) for number, note in enumerate(pitchnames))

network_input = []
network_output = []

# create input sequences and the corresponding outputs
for i in range(0, len(notes) - sequence_length, 1):
    sequence_in = notes[i:i + sequence_length]
    sequence_out = notes[i + sequence_length]
    network_input.append([note_to_int[char] for char in sequence_in])
    network_output.append(note_to_int[sequence_out])

n_patterns = len(network_input)

# reshape the input into a format compatible with LSTM layers
network_input = np.reshape(network_input, (n_patterns, sequence_length, 1))
# normalize input
n_vocab = len(set(notes))
network_input = network_input / float(n_vocab)

# Class indices for PyTorch CrossEntropyLoss (replaces Keras to_categorical)
network_output = np.array(network_output, dtype=np.int64)

return network_input, network_output, pitchnames, n_vocab

```

```

In [4]: def generate_sequence(model, network_input, pitchnames, n_vocab, device=None):
        """Generate notes using a trained PyTorch model (same sliding-window logic as t
        if device is None:
            device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

        # pick the last sequence from the input as a starting point for the prediction
        start = -1

        int_to_note = dict((number, note) for number, note in enumerate(pitchnames))

        pattern = network_input[start]
        prediction_output = []

```

```

model.eval()
with torch.no_grad():
    # generate 500 notes
    for _ in range(500):
        prediction_input = np.reshape(pattern, (1, len(pattern), 1))
        prediction_input = prediction_input / float(n_vocab)

        x = torch.FloatTensor(prediction_input).to(device)
        logits = model(x)
        index = int(torch.argmax(logits, dim=1).item())
        result = int_to_note[index]
        prediction_output.append(result)

        pattern = np.append(pattern, index)
        pattern = pattern[1 : len(pattern)]

    return prediction_output

```

```

In [5]: def create_midi(prediction_output, output_path="test_output.mid"):
        """Convert the output from the prediction to notes and create a MIDI file."""
        offset = 0
        output_notes = []

        # create note and chord objects based on the values generated by the model
        for pattern in prediction_output:
            # pattern is a chord
            if ('.' in pattern) or pattern.isdigit():
                notes_in_chord = pattern.split('.')
                notes = []
                for current_note in notes_in_chord:
                    new_note = note.Note(int(current_note))
                    new_note.storedInstrument = instrument.Piano()
                    notes.append(new_note)
                new_chord = chord.Chord(notes)
                new_chord.offset = offset
                output_notes.append(new_chord)
            # pattern is a note
            else:
                new_note = note.Note(pattern)
                new_note.offset = offset
                new_note.storedInstrument = instrument.Piano()
                output_notes.append(new_note)

        # increase offset each iteration so that notes do not stack
        offset += 0.5

        midi_stream = stream.Stream(output_notes)

        midi_stream.write("midi", fp=output_path)

```

```

In [6]: ## Import data

import os

# Locate mozart.mid (repo root or HW_04_LSTM/)

```

```

if os.path.exists("mozart.mid"):
    file_path = "mozart.mid"
elif os.path.exists(os.path.join("HW_04_LSTM", "mozart.mid")):
    file_path = os.path.join("HW_04_LSTM", "mozart.mid")
else:
    file_path = "mozart.mid"

# Fail gracefully if the file hasn't been uploaded to the Colab runtime
if not os.path.exists(file_path):
    raise FileNotFoundError(f"\n\n Could not find '{file_path}'. \nPlease make sure

network_input, network_output, pitchnames, n_vocab = generate_input_data(file_path)
num_classes = len(pitchnames)
SEQUENCE_LENGTH = network_input.shape[1]

# These arrays are the supervised training signal: input windows + next-note labels
assert network_input.shape[0] == len(network_output)

```

```

/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922: TranslateWarning: Unable to determine instrument from <music21.midi.MidiEvent SEQUENCE_TRACK_NAME, track=5, data=b'Copyright \xa9 2006 by Bernd Kr\xfcger'; getting generic Instrument

```

```

warnings.warn(
Parsing mozart.mid

```

```

/usr/local/lib/python3.12/dist-packages/music21/stream/base.py:3675: Music21DeprecationWarning: .flat is deprecated. Call .flatten() instead
    return self.iter().getElementsByClass(classFilterList)

```

Architecture: Define a Sequential model, wherein the layers are stacked sequentially and each layer has exactly one input tensor and one output tensor. Please build a LSTM neural network, in the form of a sequential model, by adding the layers to the model using the configuration below.

- LSTM | Units : 512 | Recurrent dropout : 0.3 | Return sequences : True
- LSTM | Units : 512 | Recurrent dropout : 0.3 | Return sequences : True
- Flatten
- Dense | Units : 256 | Activation : ReLU
- Dense | Units : 95 (# total notes) | Activation : Softmax

In [7]: *# Define pytorch model class (Sequential-style stack: 2x LSTM -> Flatten -> Dense -
Keras "recurrent dropout" is approximated with dropout between stacked LSTM layer*

```

class MusicLSTM(nn.Module):
    def __init__(
        self,
        sequence_length=100,
        num_classes=95,
        hidden_size=512,
        lstm_dropout=0.3,

```

```

):
    super().__init__()
    self.sequence_length = sequence_length
    self.hidden_size = hidden_size
    self.lstm = nn.LSTM(
        input_size=1,
        hidden_size=hidden_size,
        num_layers=2,
        batch_first=True,
        dropout=lstm_dropout,
    )
    self.flatten = nn.Flatten()
    flat_dim = sequence_length * hidden_size
    self.fc1 = nn.Linear(flat_dim, 256)
    self.relu = nn.ReLU()
    # Logits for nn.CrossEntropyLoss (softmax is inside the loss)
    self.fc2 = nn.Linear(256, num_classes)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = self.flatten(out)
        out = self.relu(self.fc1(out))
        out = self.fc2(out)
        return out

```

```

In [8]: def train(model, loader, criterion, optimizer, device):
    """Train the model for one epoch."""
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for inputs, labels in loader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * inputs.size(0)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(loader.dataset)
    epoch_acc = correct / total
    return epoch_loss, epoch_acc

def evaluate(model, loader, criterion, device):
    """Evaluate the model on validation or test data."""
    model.eval()
    running_loss = 0.0
    correct = 0

```

```

total = 0

with torch.no_grad():
    for inputs, labels in loader:
        inputs, labels = inputs.to(device), labels.to(device)

        outputs = model(inputs)
        loss = criterion(outputs, labels)

        running_loss += loss.item() * inputs.size(0)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

epoch_loss = running_loss / len(loader.dataset)
epoch_acc = correct / total
return epoch_loss, epoch_acc

def train_model(model, train_loader, val_loader, criterion, optimizer, device, num_epochs):
    """Train and record training / validation loss and accuracy each epoch."""
    training_loss_curve = []
    validation_loss_curve = []
    training_accuracy = []
    validation_accuracy = []

    for epoch in range(num_epochs):
        train_loss, train_acc = train(model, train_loader, criterion, optimizer, device)
        val_loss, val_acc = evaluate(model, val_loader, criterion, device)

        training_loss_curve.append(train_loss)
        training_accuracy.append(train_acc)
        validation_loss_curve.append(val_loss)
        validation_accuracy.append(val_acc)

        timestamp = datetime.now().strftime("%H:%M:%S")
        print(
            f"[{timestamp}] Epoch {epoch+1}/{num_epochs} | "
            f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
            f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}"
        )

    return training_loss_curve, validation_loss_curve, training_accuracy, validation_accuracy

# PLOT LOSS CURVES
def plot_loss(curve, dataset, model_name, output_dir="Plot JPGs"):
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    plt.figure()
    plt.plot(curve)
    plt.xlabel("Epoch")
    if dataset == "Training":
        plt.ylabel("Training Loss")
    elif dataset == "Validation":

```

```

plt.ylabel("Validation Loss")

plt.title(
    f"Categorical Cross Entropy Loss Curve for {dataset} Dataset, Model: {model_name}"
)
filename = f"{output_dir}/LossCurve_{dataset}_Dataset_Model_{model_name}.jpg"

plt.savefig(filename)
plt.close()

```

Training: The sequential model with the above configuration is finally compiled with the ADAM optimizer to minimize the categorical cross entropy loss. The model is then trained using network_input and network_output as input and target data for 50 epochs while dividing the data into a batch size of 256. Finally, the trained model can be used to generate new sequences from a given sequence of notes using the pre-defined functions introduced above.

```

In [9]: # Experiment Function
def run_experiment(model, model_name, train_loader, val_loader, test_loader, max_epochs):
    """Train with Adam + categorical cross-entropy, plot loss curves, return a result dict"""
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()))

    training_loss_curve, validation_loss_curve, training_accuracy, validation_accuracy = \
        train_model(model, train_loader, val_loader, criterion, optimizer, device, max_epochs)

    print("Training complete")

    final_test_loss, test_acc = evaluate(model, test_loader, criterion, device)
    print("Evaluation complete")

    plot_loss(training_loss_curve, "Training", model_name, output_dir=PLOT_OUTPUT_DIR)
    plot_loss(validation_loss_curve, "Validation", model_name, output_dir=PLOT_OUTPUT_DIR)
    print("Plots created")

    row = {
        "Model Name": model_name,
        "Epochs": max_epochs,
        "Training Accuracy (%)": training_accuracy[-1] * 100.0,
        "Validation Accuracy (%)": validation_accuracy[-1] * 100.0,
        "Test Accuracy (%)": test_acc * 100.0,
        "Final Test Loss": final_test_loss,
    }
    return row

```

```

In [13]: # Hyperparameters
MAX_EPOCHS = 50
BATCH_SIZE = 256

```



```

SHUFFLE_SEED = 42
torch.manual_seed(SHUFFLE_SEED)

MODEL_NAME = "LSTM_Music"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Wrap the MIDI chop-up in tensors: each batch is (windows from network_input, Labels)
X_tensor = torch.FloatTensor(network_input)
y_tensor = torch.LongTensor(network_output)
assert X_tensor.shape[0] == y_tensor.shape[0] == network_input.shape[0] == len(network_output)
dataset = TensorDataset(X_tensor, y_tensor)
print(
    f"Training data: {len(dataset)} examples from generate_input_data; "
    f"input {tuple(network_input.shape)}, targets {network_output.shape}"
)

total_n = len(dataset)
val_size = int(0.15 * total_n)
train_size = total_n - val_size
train_subset, val_subset = random_split(
    dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(SHUFFLE_SEED),
)

train_loader = DataLoader(
    train_subset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    worker_init_fn=lambda worker_id: np.random.seed(SHUFFLE_SEED),
)
val_loader = DataLoader(val_subset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = val_loader

model = MusicLSTM(
    sequence_length=SEQUENCE_LENGTH,
    num_classes=num_classes,
    hidden_size=512,
    lstm_dropout=0.3,
)

timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Run Start")

results_row = run_experiment(
    model, MODEL_NAME, train_loader, val_loader, test_loader, max_epochs=MAX_EPOCHS
)
results_df = pd.DataFrame([results_row])
display(results_df)

timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Run Finish")

# Zip and download Loss plots (Colab-friendly)
plot_zip_dir = PLOT_OUTPUT_DIR

```

```
if os.path.isdir(plot_zip_dir):
    shutil.make_archive("Plot_JPGs", "zip", plot_zip_dir)
    print("Created Plot_JPGs.zip")
    if files is not None:
        try:
            files.download("Plot_JPGs.zip")
        except Exception as e:
            print(f"Download skipped or failed: {e}")
    else:
        print(f"Plot folder not found: {plot_zip_dir}")
```

Training data: 2501 examples from generate_input_data; input (2501, 100, 1), targets (2501,)

[23:18:16] Run Start

Using device: cuda

[23:18:16] Epoch 1/50 | Train Loss: 4.0020 | Train Acc: 0.0621 | Val Loss: 3.7279 | Val Acc: 0.0613

[23:18:16] Epoch 2/50 | Train Loss: 3.7164 | Train Acc: 0.0762 | Val Loss: 3.6889 | Val Acc: 0.0667

[23:18:16] Epoch 3/50 | Train Loss: 3.6865 | Train Acc: 0.0687 | Val Loss: 3.7126 | Val Acc: 0.0667

[23:18:16] Epoch 4/50 | Train Loss: 3.6737 | Train Acc: 0.0691 | Val Loss: 3.6849 | Val Acc: 0.0667

[23:18:16] Epoch 5/50 | Train Loss: 3.6684 | Train Acc: 0.0701 | Val Loss: 3.6834 | Val Acc: 0.0613

[23:18:17] Epoch 6/50 | Train Loss: 3.6659 | Train Acc: 0.0588 | Val Loss: 3.6852 | Val Acc: 0.0667

[23:18:17] Epoch 7/50 | Train Loss: 3.6618 | Train Acc: 0.0644 | Val Loss: 3.7854 | Val Acc: 0.0533

[23:18:17] Epoch 8/50 | Train Loss: 3.6813 | Train Acc: 0.0579 | Val Loss: 3.6778 | Val Acc: 0.0720

[23:18:17] Epoch 9/50 | Train Loss: 3.6710 | Train Acc: 0.0753 | Val Loss: 3.6819 | Val Acc: 0.0667

[23:18:17] Epoch 10/50 | Train Loss: 3.6629 | Train Acc: 0.0710 | Val Loss: 3.6954 | Val Acc: 0.0613

[23:18:17] Epoch 11/50 | Train Loss: 3.6456 | Train Acc: 0.0710 | Val Loss: 3.6609 | Val Acc: 0.0720

[23:18:17] Epoch 12/50 | Train Loss: 3.6149 | Train Acc: 0.0833 | Val Loss: 3.7150 | Val Acc: 0.0667

[23:18:17] Epoch 13/50 | Train Loss: 3.6264 | Train Acc: 0.0767 | Val Loss: 3.6704 | Val Acc: 0.0667

[23:18:17] Epoch 14/50 | Train Loss: 3.6219 | Train Acc: 0.0776 | Val Loss: 3.6498 | Val Acc: 0.0560

[23:18:17] Epoch 15/50 | Train Loss: 3.5753 | Train Acc: 0.0851 | Val Loss: 3.6639 | Val Acc: 0.0640

[23:18:17] Epoch 16/50 | Train Loss: 3.5341 | Train Acc: 0.0931 | Val Loss: 3.6453 | Val Acc: 0.0640

[23:18:18] Epoch 17/50 | Train Loss: 3.5298 | Train Acc: 0.0927 | Val Loss: 3.6100 | Val Acc: 0.0640

[23:18:18] Epoch 18/50 | Train Loss: 3.4338 | Train Acc: 0.1167 | Val Loss: 3.5819 | Val Acc: 0.1280

[23:18:18] Epoch 19/50 | Train Loss: 3.3268 | Train Acc: 0.1322 | Val Loss: 3.6974 | Val Acc: 0.0933

[23:18:18] Epoch 20/50 | Train Loss: 3.2747 | Train Acc: 0.1402 | Val Loss: 3.5940 | Val Acc: 0.1200

[23:18:18] Epoch 21/50 | Train Loss: 3.1201 | Train Acc: 0.1543 | Val Loss: 3.5725 | Val Acc: 0.1253

[23:18:18] Epoch 22/50 | Train Loss: 2.9394 | Train Acc: 0.1943 | Val Loss: 3.5586 | Val Acc: 0.1200

[23:18:18] Epoch 23/50 | Train Loss: 2.7942 | Train Acc: 0.2117 | Val Loss: 3.6696 | Val Acc: 0.1227

[23:18:18] Epoch 24/50 | Train Loss: 2.5583 | Train Acc: 0.2676 | Val Loss: 3.8581 | Val Acc: 0.1173

[23:18:18] Epoch 25/50 | Train Loss: 2.3040 | Train Acc: 0.2930 | Val Loss: 3.6989 | Val Acc: 0.1573

[23:18:18] Epoch 26/50 | Train Loss: 2.0343 | Train Acc: 0.3565 | Val Loss: 4.0454 | Val Acc: 0.1093

[23:18:18] Epoch 27/50 | Train Loss: 1.7660 | Train Acc: 0.4407 | Val Loss: 4.1861 | Val Acc: 0.1360
[23:18:19] Epoch 28/50 | Train Loss: 1.5127 | Train Acc: 0.5212 | Val Loss: 4.3946 | Val Acc: 0.1973
[23:18:19] Epoch 29/50 | Train Loss: 1.2261 | Train Acc: 0.6091 | Val Loss: 4.5243 | Val Acc: 0.2613
[23:18:19] Epoch 30/50 | Train Loss: 0.9378 | Train Acc: 0.7098 | Val Loss: 4.9052 | Val Acc: 0.2800
[23:18:19] Epoch 31/50 | Train Loss: 0.6657 | Train Acc: 0.7879 | Val Loss: 5.3498 | Val Acc: 0.3227
[23:18:19] Epoch 32/50 | Train Loss: 0.5820 | Train Acc: 0.8137 | Val Loss: 5.1354 | Val Acc: 0.3493
[23:18:19] Epoch 33/50 | Train Loss: 0.3153 | Train Acc: 0.9130 | Val Loss: 5.7720 | Val Acc: 0.3733
[23:18:19] Epoch 34/50 | Train Loss: 0.1858 | Train Acc: 0.9516 | Val Loss: 6.1790 | Val Acc: 0.4000
[23:18:19] Epoch 35/50 | Train Loss: 0.0908 | Train Acc: 0.9817 | Val Loss: 6.6990 | Val Acc: 0.3893
[23:18:19] Epoch 36/50 | Train Loss: 0.0657 | Train Acc: 0.9864 | Val Loss: 6.7513 | Val Acc: 0.3920
[23:18:19] Epoch 37/50 | Train Loss: 0.0577 | Train Acc: 0.9911 | Val Loss: 7.2384 | Val Acc: 0.4027
[23:18:19] Epoch 38/50 | Train Loss: 0.0366 | Train Acc: 0.9953 | Val Loss: 7.0132 | Val Acc: 0.3947
[23:18:20] Epoch 39/50 | Train Loss: 0.0406 | Train Acc: 0.9958 | Val Loss: 7.3497 | Val Acc: 0.3973
[23:18:20] Epoch 40/50 | Train Loss: 0.0394 | Train Acc: 0.9972 | Val Loss: 7.5951 | Val Acc: 0.3973
[23:18:20] Epoch 41/50 | Train Loss: 0.0237 | Train Acc: 0.9972 | Val Loss: 7.2012 | Val Acc: 0.3947
[23:18:20] Epoch 42/50 | Train Loss: 0.0190 | Train Acc: 0.9972 | Val Loss: 7.5609 | Val Acc: 0.3947
[23:18:20] Epoch 43/50 | Train Loss: 0.0183 | Train Acc: 0.9972 | Val Loss: 7.6612 | Val Acc: 0.3973
[23:18:20] Epoch 44/50 | Train Loss: 0.0171 | Train Acc: 0.9962 | Val Loss: 7.8580 | Val Acc: 0.3920
[23:18:20] Epoch 45/50 | Train Loss: 0.0129 | Train Acc: 0.9976 | Val Loss: 7.8285 | Val Acc: 0.3947
[23:18:20] Epoch 46/50 | Train Loss: 0.0127 | Train Acc: 0.9976 | Val Loss: 7.8440 | Val Acc: 0.3947
[23:18:20] Epoch 47/50 | Train Loss: 0.0156 | Train Acc: 0.9981 | Val Loss: 7.9272 | Val Acc: 0.3947
[23:18:20] Epoch 48/50 | Train Loss: 0.0367 | Train Acc: 0.9976 | Val Loss: 7.7271 | Val Acc: 0.3893
[23:18:20] Epoch 49/50 | Train Loss: 0.0243 | Train Acc: 0.9976 | Val Loss: 7.4693 | Val Acc: 0.3947
[23:18:20] Epoch 50/50 | Train Loss: 0.0170 | Train Acc: 0.9981 | Val Loss: 7.7258 | Val Acc: 0.3920
Training complete
Evaluation complete
Plots created

	Model Name	Epochs	Training Accuracy (%)	Validation Accuracy (%)	Test Accuracy (%)	Final Test Loss
0	LSTM_Music	50	99.811853	39.2	39.2	7.725841

[23:18:21] Run Finish

Created Plot_JPGs.zip

```
In [14]: # Generate new sequence of 500 notes (uses generate_sequence + create_midi helpers)
os.makedirs("final_midi", exist_ok=True)
midi_out = os.path.join("final_midi", "generated.mid")

timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Generate Start")

prediction_output = generate_sequence(
    model, network_input, pitchnames, n_vocab, device=device
)
create_midi(prediction_output, output_path=midi_out)
print(f"Saved: {midi_out}")

timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Generate Finish")

if os.path.isdir("final_midi"):
    shutil.make_archive("final_midi", "zip", "final_midi")
    print("Created final_midi.zip")
    if files is not None:
        try:
            files.download("final_midi.zip")
        except Exception as e:
            print(f"Download skipped or failed: {e}")
```

[23:18:21] Generate Start

Saved: final_midi/generated.mid

[23:18:22] Generate Finish

Created final_midi.zip

Deliverables: Plot the loss curves for the LSTM model using the loss recorded at each step of the training process. Also, upload a sample MIDI file newly generated using the trained LSTM model containing a notes sequence of length 500. Please make sure to submit your working code files along with the generated sample MIDI file and the plots.

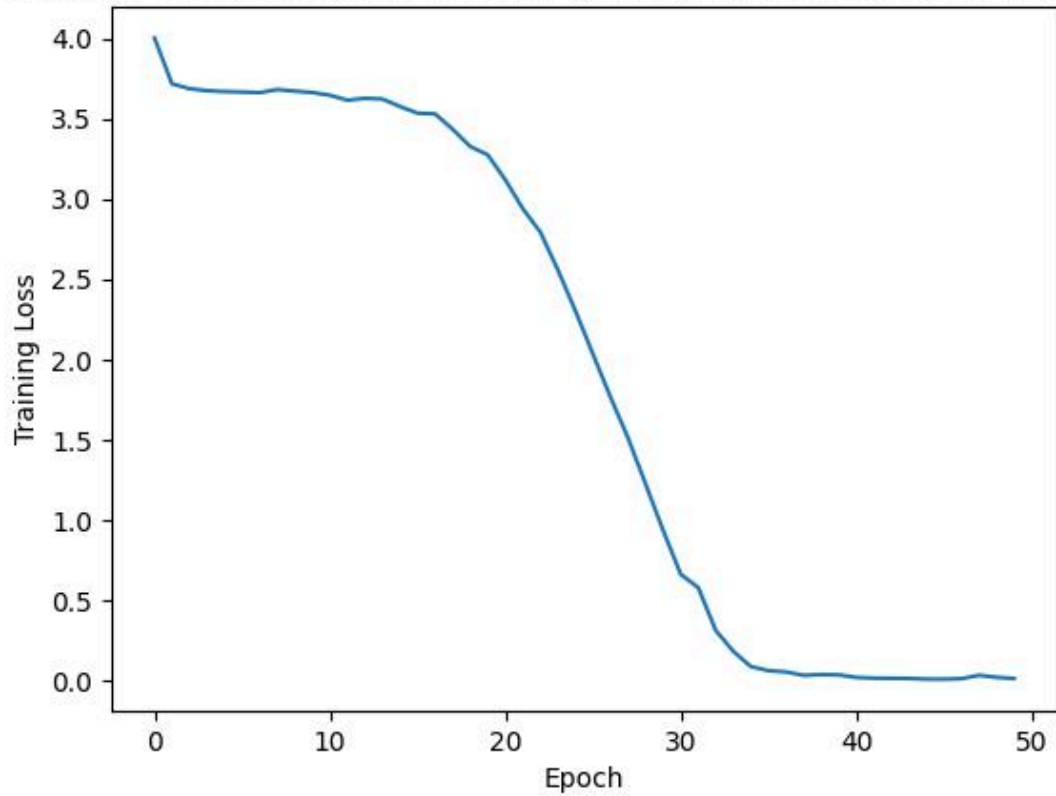
```
In [15]: # Deliverable: show every plot image saved under the plot folder
from IPython.display import Image as IPyImage

_plot_dir = Path(PLOT_OUTPUT_DIR)
if not _plot_dir.is_dir():
    print(f"No plot folder yet: {_plot_dir} (run training first)")
else:
    _exts = {".jpg", ".jpeg", ".png", ".JPG", ".JPEG", ".PNG"}
    _paths = sorted(
        p for p in _plot_dir.iterdir() if p.is_file() and p.suffix in _exts
    )
```

```
if not _paths:
    print(f"No image files in {_plot_dir}")
for _p in _paths:
    print(_p.name)
    display(IPyImage(filename=str(_p)))
```

LossCurve_Training_Dataset_Model_LSTM_Music.jpg

Categorical Cross Entropy Loss Curve for Training Dataset, Model: LSTM_Mus



LossCurve_Validation_Dataset_Model_LSTM_Music.jpg

Categorical Cross Entropy Loss Curve for Validation Dataset, Model: LSTM_Mu

